

AeroBook Premium

Building a worldwide flight booking platform,
with lessons from the Bangladesh Railway e-ticket system

Stack reviewed: Laravel 12 + Sanctum | SQL Server 2022 | React 18 + Vite | Docker

About this document

This plan is about **AeroBook Premium as it exists today**: its Laravel API, raw SQL files, SQL Server schema and React frontend. It explains how to turn AeroBook into a **worldwide flight booking platform**. The Bangladesh Railway e-ticket site (eticket.railway.gov.bd) is used only as a *source of good ideas*. The features it does well are borrowed and adapted to how air travel works. It is not copied.

- **Section 1:** what the project does today, including what works, what is empty, and what the README claims but the code does not do.
- **Section 2:** the features worth borrowing from the Railway e-ticket platform, and how each one changes for flights.
- **Section 3:** flight-specific features a worldwide platform needs, including the **domestic flight rule** (no visa, passport or NID) and the **colour-coded seat map**.
- **Sections 4-8:** bugs to fix, the new database design, SQL procedures and triggers, frontend pages and a phased roadmap.
- **Section 9:** features from leading booking platforms (Google Flights, Skyscanner, Kayak, Hopper, Trip.com, Expedia, Kiwi.com, GoZayaan, ShareTrip) that would add value to AeroBook.

Summary of the verdict

Area	Current state	Priority
Core booking flow	One click books the <i>first free seat</i> on a one-way flight. There is no seat choice, no traveller details, no payment and no ticket.	Critical
Worldwide readiness	Only 4 domestic Bangladeshi airports. No time zones, currencies, fares, cabin classes or return trips.	Critical
Database features	The README promises triggers and stored procedures, but none exist anywhere in the project.	Critical
Traveller data	The passport is tied <i>uniquely</i> to the account, so a user can never book for family or colleagues.	High

Area	Current state	Priority
Frontend	About 22 component/page files are empty (0 bytes) . Tailwind classes are used but Tailwind is not installed.	High
Security	Fake social-login form collects passwords. Hard-coded sa password. User enumeration. Debug mode on.	High
Admin	Only a read-only dashboard. Admins cannot manage routes, flights, fares, aircraft or seat maps.	Medium

1. What the project is today

1.1 Architecture

Layer	Technology	Notes
Frontend	React 18, React Router 6, Axios, Vite 5	Single-page app. API URL hard-coded to <code>http://localhost:8000/api</code> .
Backend API	Laravel 12, Sanctum bearer tokens	5 controllers. Queries are mostly raw SQL loaded from <code>database/sql/*.sql</code> .
Database	SQL Server 2022 (Docker)	11 business tables. Seat locking uses <code>WITH (UPDLOCK, ROWLOCK)</code> .
Infra	<code>docker-compose: app, nginx, sqlserver, sqlserver-init</code>	One-shot init container creates the aerobook database.

1.2 Implemented API endpoints

Method & path	Purpose	Auth
GET <code>/api/airports</code>	List airports (dropdown data)	public
GET <code>/api/flights/search</code>	One-way search by origin city, destination city, date and passenger count	public
POST <code>/api/auth/register</code> login admin-login	Account creation and login (throttled)	public
POST <code>/api/auth/password/request-code</code> reset	6-digit emailed reset code, valid for 10 minutes	public
GET/PUT <code>/api/auth/me</code> , <code>/auth/profile</code> ; POST <code>/auth/logout</code>	Profile management	user
GET <code>/api/bookings</code> , <code>/bookings/{id}</code> ; POST <code>/bookings</code>	List, view and create bookings	user
GET <code>/api/admin/dashboard</code>	KPIs, occupancy and revenue as one FOR JSON query	admin

1.3 Things done well (keep these)

- **Race-condition-safe seat grab.** `booking_available_seat.sql` selects the seat `WITH (UPDLOCK, ROWLOCK)` inside a transaction, so two users cannot take the same seat. This is the most valuable idea in the project, and the seat-hold design in Section 6 builds on it.
- **Readable SQL kept apart from the PHP code** in `database/sql/`. This makes the database logic easy to present.
- **Admin dashboard in a single query** using `FOR JSON PATH`.
- **Separate admins table**, rate-limited auth routes, hashed reset codes with a 10-minute expiry, and a strong password rule.

- Aircraft have **flight hours and a maintenance threshold**. This is a good base for the fleet features airlines need.
- The passengers table already has **frequent_flyer_points**, a ready starting point for a loyalty programme.

1.4 Placeholders and broken promises

These files exist but are **completely empty (0 bytes)**. They show the flight journey that was planned but never built:

Folder	Empty files
src/pages	FlightSearchPage, SeatSelectionPage, PaymentPage, BookingConfirmationPage
src/pages/admin	AnalyticsPage, FleetManagementPage, FlightManagementPage
src/components/booking	SeatMap, SeatCell, FlightSearchForm, PaymentForm, BookingSummary
src/components/admin	AircraftForm, AircraftTable, FlightScheduler, MaintenanceLogTable
src/components/common	Navbar, Footer, Button, Loader, ProtectedRoute
src/components/dashboard	FleetHealthPanel
src/routes, src/hooks	AppRouter.jsx, useAuth.js, useSeatAvailability.js

The README claims features that do not exist. It says *"Triggers update aircraft maintenance logs automatically"* and *"Stored procedures generate occupancy and revenue reports"*. There is no CREATE TRIGGER or CREATE PROCEDURE anywhere in the repository. Section 6 gives ready-to-use SQL that makes these claims true.

2. What to borrow from the Bangladesh Railway e-ticket platform

The Railway e-ticket site handles very high demand for a limited number of seats. Its strengths are **fairness**, **reliability under load** and **paperless tickets**. These ideas carry over well to flights. Its rules, however, are built for one country's domestic trains, so every borrowed feature below is **adapted** for an international airline platform. Some Railway rules are deliberately **not** copied.

Railway e-ticket feature	Adapted for AeroBook (worldwide flights)
Verified accounts (NID + date of birth + mobile OTP)	Use email verification plus an optional phone OTP for security. Store saved traveller profiles (name as on the ID, date of birth, nationality, and a passport and/or NID) to reuse at checkout. An NID is accepted for domestic flights (Section 3.1). International flights need a passport. An account holder can book for other people.
Visual seat map with live availability (per coach)	A cabin seat map per aircraft type (e.g. B737-800: 3-3 layout, A350: 3-3-3). Seat types include window/aisle/middle, exit row , extra legroom and bassinet. Paid seats show a surcharge. Occupied seats are greyed out and refresh live.
Seat hold with a payment countdown	Hold the chosen fare and seats for about 15 minutes while the traveller fills in details and pays. A visible timer runs, and a background job releases expired holds. This fixes AeroBook's instant 'confirmed' without payment.
Limit per purchase (a few seats per transaction)	Use the airline norm instead: at most 9 passengers per booking , and infants no more than adults (an infant sits on an adult's lap). Groups larger than 9 go through a group request form.
Instant e-ticket by email/SMS, shown on a phone	Issue a PNR (6-character booking reference) and one e-ticket number per passenger , emailed as a PDF. After online check-in, issue a boarding pass with a QR/barcode that works on a phone.

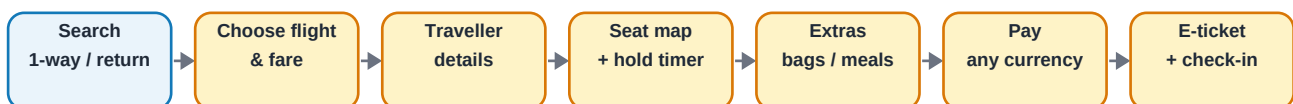
Railway e-ticket feature	Adapted for AeroBook (worldwide flights)
Purchase history: download, cancel, refund	A Manage Booking page, reachable by logging in <i>or</i> by PNR + last name (the airline standard). It covers ticket download, change and cancel under the fare rules , and shows refund status.
Verify ticket page	A public Booking lookup and Flight status page (on time, delayed, gate, cancelled).
Bangla / English language switch	Full multi-language support (English first, then e.g. Bangla, Arabic, Chinese, with right-to-left layout for Arabic). Add multi-currency display and times in each airport's local time zone .
Local payment methods (bKash, Nagad, Rocket, cards) and a clear fare breakdown	Use a global gateway (e.g. Stripe/PayPal sandbox) for cards, plus regional wallets where the user is (e.g. bKash/Nagad via SSLCommerz for Bangladesh). Show base fare + taxes + surcharges + seat/bag extras per passenger.
High-traffic protection: CAPTCHA, rate limits, one active session	Keep the rate limits and single session. Add a CAPTCHA on sign-up, login and search. Add idempotency keys on payment so a double click never charges twice. Optionally add a waiting-room page for flash sales.

2.1 Railway rules deliberately not copied

- **NID for every traveller.** The NID stays, but only as an **option for domestic flights**. International travellers are identified by **passport** data (Section 3.1).
- **A short advance-sale window.** Airlines normally sell up to about **a year ahead**, and close sales a few hours before departure.
- **Coaches, trains and fixed stations.** These become **aircraft, cabins and IATA airports**, with connections between flights.
- **A single-currency, domestic-only design.** Every price needs a currency, and every time needs a time zone.

3. Flight-specific features a worldwide platform needs

The booking journey for AeroBook, with the Railway-inspired steps built in. Only search exists today:



 exists (partly) in AeroBook to build

Feature	What it means	Why it matters
Trip types	One-way, return and later multi-city . The Round Trip radio already exists on HomePage but does nothing.	Most flight searches are return trips.
Airport search	Type-ahead on city, airport name or IATA code (DAC, LHR, DXB, JFK...). A worldwide airport table with country and time zone.	A dropdown of 4 cities cannot scale to thousands of airports.
Flexible dates	A fare calendar showing the lowest price for each day around the chosen date.	A common way travellers pick a cheaper day.
Connections	Itineraries with 1-2 stops (e.g. DAC to DXB to LHR), each with a minimum connection time. Show total duration and layover.	Many worldwide routes have no direct flight.
Cabin classes & fare families	Economy / Premium Economy / Business / First, each with fare families (e.g. Saver, Standard, Flex). These differ in baggage, change fee and refundability .	This replaces the current 'business' and 'economy' labels on seats, which have no prices.

Feature	What it means	Why it matters
Passenger types	Adult (12+), Child (2-11) and Infant (under 2), with different fares and rules.	Required for correct pricing and safety rules.
Travel documents	Depends on the trip. Domestic: a passport or NID, and no visa (Section 3.1). International: full passport data (APIS), with a warning if the passport expires within 6 months of travel.	International flights must send passport data to border authorities. Domestic flights should stay simple.
Ancillaries	Extra baggage, meals, seat selection, priority boarding and travel insurance, added before payment.	A major source of airline revenue, and a good feature to show.
Online check-in	Opens 48-24 hours before departure. The traveller confirms documents and receives a QR boarding pass.	Completes the paperless idea borrowed from the Railway site.
Flight status & notifications	Email (and optionally SMS) alerts for booking confirmation, schedule changes, delays, gate changes and check-in opening.	Travellers expect proactive updates.
Loyalty programme	Earn points (the frequent_flyer_points column already exists) based on distance and cabin. Show tiers on the profile.	Brings customers back. The data model is already started.
Promo codes	Percentage or fixed discounts with validity dates, routes and usage limits.	A simple, testable pricing feature.
Time zones	Store all times in UTC (DATETIMEOFFSET) and display them in the local time of each airport. Handle overnight arrivals (+1 day).	A 10:00 flight from Dhaka and a 10:00 flight from London are 6 hours apart.

3.1 Domestic flights: no visa, passport or NID accepted

A flight is **domestic** when the origin and destination airports are in the **same country** (e.g. Dhaka to Chattogram, Dhaka to Cox's Bazar). Nobody crosses a border, so **no visa is needed**. The traveller can identify themselves with **either a passport or a national ID card (NID)**. This makes domestic booking as quick as the Railway e-ticket experience, while international trips keep the stricter airline rules. AeroBook's current seed data (Dhaka, Chittagong, Sylhet, Rajshahi) is entirely domestic, so this rule is useful straight away.

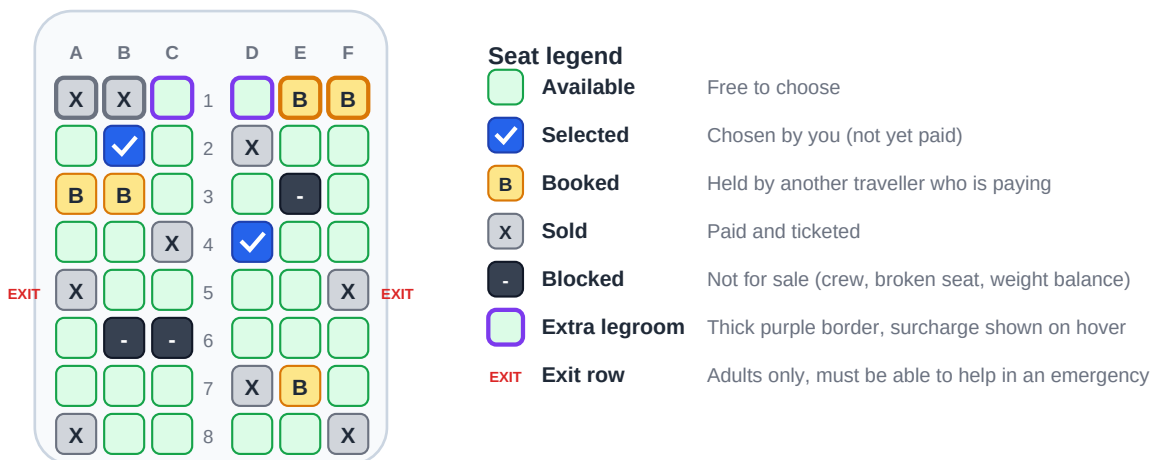
Rule	Domestic flight	International flight
How it is detected	origin.country_code = destination.country_code on every segment	Any segment crosses a border (a mixed itinerary counts as international)
Accepted document	Passport or NID (the traveller chooses)	Passport only
Visa	Not required. No visa notice is shown.	Traveller confirms they hold the required visa/entry permit. AeroBook shows an 'entry requirements' reminder.
Document validity	Passport must be unexpired on the travel date. The NID has no expiry check.	Passport must be valid for 6 months after the travel date (common airline rule)
Data collected	Name as on the ID, date of birth, document type + number	Full APIS: name, DOB, gender, nationality, passport number, issuing country, expiry
Who can use an NID	Anyone holding the NID of the country the flight is in (a Bangladeshi NID for a flight inside Bangladesh). Foreign visitors use their passport.	Not applicable
Check-in / boarding	The same document is shown at the airport. Its number is printed on the boarding pass.	The passport is checked at check-in and by immigration

How to build it

- **Database:** add a computed column on routes: `is_domestic AS CASE WHEN o.country_code = d.country_code THEN 1 ELSE 0 END` (or set it when the route is saved). Replace `passport_no` in `booking_passengers` with `document_type ('PASSPORT', 'NID')`, `document_number` (encrypted), `document_country` `CHAR(2)` and `document_expiry` (NULL for an NID).
- **Rule in the database:** a trigger `trg_document_rule` (Section 6.4) rejects an NID on any booking that has an international segment, and rejects an NID whose country is not the flight's country.
- **API:** `GET /flights/search` returns `is_domestic`. The traveller-details endpoint validates with Laravel rules like `required_if:is_domestic, false for passport fields`. Bangladeshi NIDs are 10, 13 or 17 digits, so check that format.
- **Frontend:** on the traveller form for a domestic trip, show a **Passport | National ID** toggle and hide the visa notice. International trips show passport fields and a 'Check visa and entry requirements' reminder. Search results show a small **Domestic / International** badge.
- **Profile:** saved travellers can store both a passport and an NID. Checkout picks the right one automatically.

3.2 Seat map colour legend: available, selected, booked, sold

The Railway e-ticket seat map uses **colour-coded seats with a legend**, so a traveller can see at a glance which seats they can take, which ones they have picked, and which are already gone. AeroBook's seat map (the empty `SeatMap.jsx` and `SeatCell.jsx` files) should work the same way, drawn as an aircraft cabin. Example of a 3-3 economy cabin:



Status	Colour + symbol	Stored where	Can the user click it?
Available	Light green, empty	<code>seats.status = 'available'</code>	Yes. It turns Selected .
Selected	Blue with a tick	Only in the browser (React state) until 'Continue'. After the hold, the API marks the user's own held seats as <i>selected</i> .	Yes. Clicking again deselects it.
Booked	Amber with 'B'	<code>seats.status = 'booked' + a seat_holds row</code> (someone else is paying, max 15 min)	No. It may turn green again if their hold expires.
Sold	Grey with 'X'	<code>seats.status = 'sold'</code> after <code>sp_confirm_booking</code>	No. It is permanent unless the ticket is cancelled.
Blocked	Dark grey with '-'	<code>seats.status = 'blocked'</code> , set by an admin	No. The seat is never for sale.

Status	Colour + symbol	Stored where	Can the user click it?
Extra legroom / Exit row	Purple border / red 'EXIT' label	seat_layouts.seat_type + surcharge	Yes, if available. Shows the price and exit-row conditions first.

Behaviour rules

- **Selection limit:** the user can select exactly as many seats as there are seated travellers (infants have no seat). A counter shows *'2 of 3 seats selected'*, and **Continue** stays disabled until all are chosen.
- **One flight at a time:** on a return or connecting trip, a tab per flight (e.g. DAC-DXB, DXB-LHR) keeps a separate legend state for each.
- **Live updates:** useSeatAvailability reloads the seat statuses every 5-10 seconds. A seat another person holds turns **amber**, a paid one turns **grey**, and an expired hold turns **green** again without refreshing the page.
- **Server is the judge:** if two people pick the same green seat, sp_hold_seats lets only one win. The other sees *'This seat was just taken'*, and it turns amber on their screen.
- **Hover/tap details:** seat number, type (window/aisle), and the price or 'Included in your fare'.
- **Accessible:** never colour only. Every state has a symbol (tick, B, X, -), a text label for screen readers (aria-label="Seat 12A, available, window"), and keyboard navigation with the arrow keys.

Example: SeatCell.jsx + CSS classes

```
// SeatCell.jsx: one seat on the map; status comes from GET /flights/{id}/seatmap
const MARK = { selected: '✓', booked: 'B', sold: 'X', blocked: '-' };
export default function SeatCell({ seat, selected, onToggle }) {
  const status = selected ? 'selected' : seat.status; // available | booked | sold | blocked
  const clickable = status === 'available' || status === 'selected';
  return (
    <button className={`seat seat--${status} ${seat.type === 'extra_legroom' ? 'seat--xl' : ''}`}
      disabled={!clickable} onClick={() => onToggle(seat)}
      aria-label={`Seat ${seat.number}, ${status}, ${seat.position}`}>
      {MARK[status] ?? ''}
    </button>
  );
}
/* App.css */
.seat--available { background:#DCFCE7; border:1px solid #16A34A; }
.seat--selected { background:#2563EB; color:#fff; }
.seat--booked { background:#FDE68A; border:1px solid #D97706; cursor:not-allowed; }
.seat--sold { background:#D1D5DB; color:#374151; cursor:not-allowed; }
.seat--blocked { background:#374151; color:#fff; cursor:not-allowed; }
.seat--xl { border:2px solid #7C3AED; }
```

4. Bugs and problems found in the code

Fix these first. They are small, but examiners notice them.

4.1 Correctness

#	Where	Problem	Fix
1	BookingController::store	Always books 1 seat (the first free one) and sets status ' confirmed ' without any payment. It ignores the passenger count the user chose.	Replace it with the hold, pay and confirm flow in Section 5.2.
2	passengers table + RegisterRequest	passport and email are UNIQUE and tied to the user. Nobody can book for a child, a partner or a colleague.	Move traveller details into booking_passengers (one row per traveller per booking).
3	BookingController::store	If the user has no passenger row, it inserts passport 'AUTO-{id}', which is fake data.	Collect real traveller details at checkout.
4	Whole schema	No prices or currencies anywhere. The payments table is never written to, yet the dashboard shows revenue.	Add fares (with currency) and write the payments rows.
5	flights table	origin and destination are free-text <i>city names</i> , not airport foreign keys. There is no flight number.	Use origin_airport_id and destination_airport_id foreign keys, plus flight_no (e.g. AB 201).
6	flights.departure / arrival	Stored as a plain datetime with no time zone. International times would be wrong.	Use DATETIMEOFFSET in UTC, and store a time zone on each airport.
7	FlightController::search	CAST(f.departure AS DATE) = :date cannot use an index. There is no input validation.	Use departure >= :d AND departure < DATEADD(day, 1, :d) and add an index. Validate with \$request->validate().
8	HomePage.jsx	The Round Trip / One Way radios and the swap icon do nothing. The 'Flights' link is href="#".	Implement the return search and wire up the swap.
9	MyBookingsPage, App.jsx	Use Tailwind classes, but Tailwind is not installed, so they have no effect.	Install Tailwind, or move those styles to App.css.
10	Seeder	4 Bangladeshi airports, 6 seats per flight, all flights on tomorrow's date, and the Airbus is never used.	Seed about 30 international airports, real seat layouts and 30+ days of schedules.
11	RegisterRequest	email:rfc,dns needs internet DNS. Registration fails when offline, for example during a demo.	Use email:rfc.
12	Frontend	The API URL is hard-coded twice (axiosClient.js and flights.js). Prices show as \$ although there is no currency.	Use import.meta.env.VITE_API_URL and one currency formatter.
13	Repo hygiene	frontend/dist and composer.phar are committed. RUN_PROJECT.txt uses another person's path.	Add them to .gitignore and use relative paths.

4.2 Security

#	Issue	Why it matters	Fix
S1	SocialLoginPage shows a Facebook/YouTube/Instagram-branded form that asks for that platform's password.	This looks exactly like a phishing page. It must not be shown or demoed.	Delete it. For social login, use real OAuth via Laravel Socialite (Google, Apple, Facebook).
S2	Login returns <i>'Invalid credentials'</i> vs <i>'Wrong password'</i> . Password reset returns 404 'No account found'.	Attackers can find out which emails are registered.	Always return the same generic message.
S3	SQL Server sa password hard-coded in docker-compose.yml, .env.example and RUN_PROJECT.txt. The app connects as sa.	sa has full control of the server.	Read it from .env. Use a least-privilege login that only has EXECUTE on the procedures plus the SELECTs it needs.
S4	APP_DEBUG=true by default	Stack traces leak paths and SQL.	Set false outside local development.
S5	password_reset_tokens is keyed by email only, but is shared by the users and admins tables.	The same email in both tables means one reset code overwrites the other.	Add an account_type column to the key.
S6	AdminMiddleware trusts role === 'admin' on any token owner, and the users table still has a role column.	A users row with role='admin' would get admin access.	Check \$user instanceof Admin.
S7	Tokens are stored in localStorage and never expire. Passport data is stored in plain text.	Tokens can be stolen through XSS. A database leak would expose identity documents.	Set sanctum.expiration. Encrypt passport numbers (Laravel 'encrypted' cast).

5. Target design

5.1 New and changed tables

Keep the existing tables and add or modify the following. Names follow the current naming style.

Table	Key columns	Purpose
airports (modify)	iata CHAR(3) UNIQUE, name, city, country_code CHAR(2), timezone (e.g. 'Asia/Dhaka'), lat, lon	Worldwide airport search and local times
routes (new)	origin_airport_id, destination_airport_id, distance_km, min_connection_minutes, is_domestic	Network map. Distance drives loyalty points and CO2 estimates. is_domestic drives the document rule.
flights (modify)	flight_no, route_id FK, aircraft_id, departure_utc / arrival_utc DATETIMEOFFSET, status, sale_closes_at	Proper foreign keys and time zones
cabin_classes (new)	code (Y, W, J, F), name	Economy, Premium Economy, Business, First
fares (new)	flight_id, cabin_class_id, fare_family (SAVER, STANDARD, FLEX), base_fare, currency CHAR(3), baggage_kg, refundable, change_fee	Price and rules per option
seat_layouts (new)	aircraft_id, row_no, seat_letter, cabin_class_id, seat_type (window, aisle, exit, extra_legroom), surcharge	Draws the seat map once per aircraft type
seats (modify)	+ layout_id FK, status ('available','booked','sold','blocked') replacing is_booked	Live seat state, the same as the seat-map legend (Section 3.2)
seat_holds (new)	seat_id UNIQUE, booking_id, expires_at	The Railway-style timed hold

Table	Key columns	Purpose
bookings (modify)	+ pnr CHAR(6) UNIQUE, user_id, trip_type, contact_email, currency, total_amount, status	One booking = one PNR, 1-9 travellers
booking_segments (new)	booking_id, flight_id, fare_id, direction (outbound/return), segment_no	Return trips and connections
booking_passengers (new)	booking_id, pax_type (ADT/CHD/INF), first/last name, dob, nationality, document_type (PASSPORT/NID), document_number (encrypted), document_country, document_expiry, ticket_number	Travellers per booking (fixes bug #2). Supports the domestic NID rule.
passenger_seats (new)	booking_passenger_id, segment_id, seat_id	A seat for each traveller on each flight
ancillaries (new)	booking_passenger_id, segment_id, type (bag, meal, priority), price, currency	Extras added before payment
payments (modify)	+ method, currency, gateway_txn_id UNIQUE, idempotency_key UNIQUE	Real payment tracking with no double charges
refunds, checkins (new)	refunds: amount, status. checkins: passenger, segment, boarding_pass_code, checked_in_at	Cancellation and online check-in
saved_travellers (new)	user_id, the same identity fields as booking_passengers	Reuse family members' details at checkout
exchange_rates, promo_codes, audit_log (new)	currency pair + rate; code + rules; table, row, action, old values	Multi-currency display, discounts, and a history for triggers

5.2 Booking state machine

Each move happens inside **one transaction**, run by a stored procedure:

From	Event	To	Effect
(none)	Traveller chooses fares and seats and presses Continue	held	Seats set to 'booked' (amber for everyone else). seat_holds rows expire in 15 minutes. The countdown shows in the UI.
held	Payment confirmed by the gateway	ticketed	Seats set to 'sold' (grey). PNR and e-ticket numbers issued. Email sent. Loyalty points added.
held	Timer expires or the user leaves	expired	Seats back to 'available' (green) via the scheduled job
ticketed	User changes the flight (if the fare allows)	ticketed	Old seat freed, new seat booked, fare difference + change fee charged
ticketed	User cancels	cancelled / refunded	Refund = the amount allowed by the fare family (e.g. Saver: taxes only, Flex: full)
ticketed	Online check-in	checked_in	Boarding pass with QR code issued

5.3 New API endpoints

Endpoint	Description
GET /airports?q=lon	Type-ahead airport search by city, name or IATA code
GET /flights/search?from&to&depart&return&adults&children&infants&cabin	One-way or return results, including 1-stop connections, with fares per fare family
GET /flights/fare-calendar?from&to&month	Lowest fare for each day
GET /flights/{id}/seatmap	Layout + live status + surcharge per seat

Endpoint	Description
POST /bookings/hold	Calls sp_hold_seats. Returns booking_id + expires_at.
PUT /bookings/{id}/passengers, /ancillaries	Save travellers (passport or NID, validated against the domestic/international rule) and extras
POST /payments/{booking}/init, POST /payments/webhook	Start payment (with idempotency key). The gateway callback calls sp_confirm_booking.
GET /manage-booking?pnr&last_name	Airline-style booking retrieval without logging in
POST /bookings/{id}/cancel, /change	Applies the fare rules and creates a refund or fare difference
POST /checkin/{pnr}, GET /boarding-pass/{id}.pdf	Online check-in and QR boarding pass
GET /flight-status?flight_no&date	Public flight status
Admin: /admin/airports, /routes, /flights, /fares, /aircraft, /seat-layouts, /promo-codes, /reports	Full CRUD and reports that fill the empty admin pages

6. Database logic: stored procedures and triggers

The README says the system is "handled natively at the database layer". These scripts make that true, and they are the most interesting parts to present: concurrency control, triggers and reporting in T-SQL.

6.1 sp_hold_seats: concurrency-safe seat hold (the Railway idea, for flights)

```

CREATE OR ALTER PROCEDURE dbo.sp_hold_seats
    @user_id INT, @flight_id INT, @seat_ids NVARCHAR(200), -- e.g. '12,13,14'
    @hold_minutes INT = 15, @booking_id INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    DECLARE @ids TABLE (seat_id INT PRIMARY KEY);
    INSERT @ids SELECT CAST(value AS INT) FROM STRING_SPLIT(@seat_ids, ',');

    IF (SELECT COUNT(*) FROM @ids) NOT BETWEEN 1 AND 9          -- airline limit per booking
        THROW 50001, 'A booking can contain 1 to 9 seated passengers.', 1;

    IF EXISTS (SELECT 1 FROM dbo.flights
        WHERE id = @flight_id AND (status <> 'scheduled' OR sale_closes_at < SYSDATETIMEOFFSET()))
        THROW 50003, 'Sales for this flight are closed.', 1;

    BEGIN TRAN;
        -- lock the chosen seats; a concurrent request for the same seat waits here
        IF EXISTS (SELECT 1 FROM dbo.seats s WITH (UPDLOCK, HOLDLOCK)
            JOIN @ids i ON i.seat_id = s.id
            WHERE s.flight_id <> @flight_id OR s.status <> 'available')
            THROW 50002, 'One or more selected seats are no longer available.', 1;

        INSERT dbo.bookings (user_id, status, created_at)
        VALUES (@user_id, 'held', SYSDATETIMEOFFSET());
        SET @booking_id = SCOPE_IDENTITY();

        UPDATE s SET status = 'booked' FROM dbo.seats s JOIN @ids i ON i.seat_id = s.id;
        INSERT dbo.seat_holds (seat_id, booking_id, expires_at)
        SELECT seat_id, @booking_id, DATEADD(MINUTE, @hold_minutes, SYSDATETIMEOFFSET()) FROM @ids;
    COMMIT;
END

```

6.2 sp_release_expired_holds: runs every minute

```

CREATE OR ALTER PROCEDURE dbo.sp_release_expired_holds AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRAN;
        UPDATE s SET status = 'available'
        FROM dbo.seats s JOIN dbo.seat_holds h ON h.seat_id = s.id
        WHERE h.expires_at < SYSDATETIMEOFFSET();

        UPDATE b SET status = 'expired'

```

```

FROM dbo.bookings b
WHERE b.status = 'held'
    AND EXISTS (SELECT 1 FROM dbo.seat_holds h
                WHERE h.booking_id = b.id AND h.expires_at < SYSDATETIMEOFFSET());

DELETE dbo.seat_holds WHERE expires_at < SYSDATETIMEOFFSET();
COMMIT;
END
-- Laravel scheduler: Schedule::call(fn() => DB::statement('EXEC dbo.sp_release_expired_holds'))->everyMinute();

```

6.3 Trigger: flight hours and automatic maintenance (makes the README claim true)

```

CREATE OR ALTER TRIGGER dbo.trg_flight_completed ON dbo.flights AFTER UPDATE AS
BEGIN
    SET NOCOUNT ON;
    UPDATE a SET total_flight_hours = a.total_flight_hours
        + DATEDIFF(MINUTE, i.departure_utc, i.arrival_utc) / 60.0
    FROM dbo.aircraft a
    JOIN inserted i ON i.aircraft_id = a.id
    JOIN deleted d ON d.id = i.id
    WHERE i.status = 'landed' AND d.status <> 'landed';

    INSERT dbo.maintenance_logs (aircraft_id, [date], description, status, created_at)
    SELECT a.id, CAST(GETUTCDATE() AS DATE),
        CONCAT('Auto: ', a.total_flight_hours, ' h reached threshold ', a.maintenance_threshold),
        'scheduled', SYSDATETIMEOFFSET()
    FROM dbo.aircraft a
    WHERE a.id IN (SELECT aircraft_id FROM inserted)
        AND a.total_flight_hours >= a.maintenance_threshold
        AND NOT EXISTS (SELECT 1 FROM dbo.maintenance_logs m
                        WHERE m.aircraft_id = a.id AND m.status IN ('scheduled','in_progress'));
END

```

6.4 Trigger: domestic flights accept passport or NID, international needs a passport

```

CREATE OR ALTER TRIGGER dbo.trg_document_rule ON dbo.booking_passengers AFTER INSERT, UPDATE AS
BEGIN
    SET NOCOUNT ON; -- segments are created first by sp_hold_seats, so they exist here
    -- 1) an NID is only valid when EVERY segment is domestic, inside the NID's own country
    IF EXISTS (SELECT 1 FROM inserted p
                JOIN dbo.booking_segments bs ON bs.booking_id = p.booking_id
                JOIN dbo.flights f ON f.id = bs.flight_id
                JOIN dbo.routes r ON r.id = f.route_id
                JOIN dbo.airports o ON o.id = r.origin_airport_id
                WHERE p.document_type = 'NID'
                AND (r.is_domestic = 0 OR o.country_code <> p.document_country))
    BEGIN
        ROLLBACK;
        THROW 50010, 'A national ID can only be used for domestic flights. Please use a passport.', 1;
    END

    -- 2) passport validity: travel date for domestic, +6 months for international (no visa check on domestic)
    IF EXISTS (SELECT 1 FROM inserted p
                JOIN dbo.booking_segments bs ON bs.booking_id = p.booking_id
                JOIN dbo.flights f ON f.id = bs.flight_id
                JOIN dbo.routes r ON r.id = f.route_id
                WHERE p.document_type = 'PASSPORT'
                AND p.document_expiry < CASE WHEN r.is_domestic = 1
                                            THEN CAST(f.departure_utc AS DATE)
                                            ELSE DATEADD(MONTH, 6, CAST(f.departure_utc AS DATE)) END)
    BEGIN
        ROLLBACK;
        THROW 50011, 'The passport is not valid long enough for this trip.', 1;
    END
END

```

6.5 More procedures, triggers and views

Object	What it does
sp_confirm_booking(@booking_id, @txn_id, @amount, @currency)	Checks the hold is still valid and the amount matches. Sets the seats to 'sold', generates the PNR and ticket numbers, inserts the payment and adds loyalty points. All in one transaction.

Object	What it does
sp_cancel_booking(@booking_id, @user_id)	Reads the fare family rules (refundable, change_fee), frees the seats and inserts the refund.
sp_search_itineraries(@from, @to, @date, @pax, @cabin)	Direct flights plus 1-stop connections that respect min_connection_minutes
sp_occupancy_report / sp_revenue_report	Load factor per route and cabin, and revenue by day, route, currency or payment method. Replaces the inline dashboard SQL, as the README promises.
trg_audit_bookings / trg_audit_payments	Write every change into audit_log with the old values as JSON
trg_block_overbooking	Safety net: rolls back if the booked seats would exceed the aircraft capacity
trg_infant_rule	Rejects a booking where infants outnumber adults
trg_document_rule	Domestic: passport or NID allowed, no visa. Rejects an NID on any booking with an international segment, or an NID from a different country.
trg_fare_history	Copies every fare price change into fare_history. This powers the price graph and price alerts in Section 9.
vw_flight_availability	Seats left per flight and cabin. Keeps the search query short.
Indexes	flights(route_id, departure_utc); seats(flight_id, status); bookings(pnr); booking_passengers(last_name); a filtered index on seat_holds(expires_at)

7. Frontend pages

Route	Page	Behaviour
/	Home + search	Trip-type tabs (one-way/return), airport type-ahead with IATA codes, a working swap button, a date range picker, a travellers picker (adults/children/infants + cabin). Links to Manage Booking and Flight Status.
/search	FlightSearchPage	Results with filters (stops, times, price) and sorting (cheapest, fastest). Each flight expands to fare-family cards (Saver/Standard/Flex) showing baggage and refund rules. A fare calendar strip at the top.
/booking/:id/travellers	Traveller details	One form per traveller. For a domestic trip: a Passport / NID toggle and no visa notice. For an international trip: passport fields plus an entry-requirements reminder. 'Use a saved traveller' option. Contact email/phone.
/booking/:id/seats	SeatSelectionPage + SeatMap	Aircraft cabin drawing with the colour legend from Section 3.2 (available, selected, booked, sold, blocked) and a surcharge legend. One flight tab per segment. The hold countdown is visible from here to payment.
/booking/:id/extras	Extras	Bags, meals, insurance. A running total per passenger.
/booking/:id/pay	PaymentPage	Price breakdown (fare, taxes, extras), currency selector, card or wallet options.
/booking/:id/confirmed	BookingConfirmationPage	PNR, ticket numbers, e-ticket PDF download, 'Add to calendar'.
/manage	Manage Booking	Log in or use PNR + last name. Change, cancel, add extras, check in, download the boarding pass.
/flight-status	Flight Status	Search by flight number or route and date

Route	Page	Behaviour
/my-bookings, /profile	Account	Upcoming/Past/Cancelled tabs, saved travellers, loyalty points and tier.
/admin/*	Admin	Fill the empty pages: FlightManagement (routes, schedules, fares), FleetManagement (aircraft, seat layouts, maintenance), Analytics (reports from sp_*).

7.1 Frontend clean-up checklist

- Move the routes into the empty routes/AppRouter.jsx. Create one shared Navbar (it is currently copy-pasted into several pages).
- Choose **one** styling approach: install Tailwind, or convert its classes to App.css.
- Use one Axios client. flights.js uses fetch() with its own hard-coded URL.
- Add **react-i18next** for languages, **Intl.NumberFormat** for currencies, and **Intl.DateTimeFormat** with the airport's time zone for times.
- Implement useSeatAvailability to refresh the seat map every few seconds, so seats held by others turn amber and sold seats turn grey (Section 3.2).
- Make it responsive for phones, add accessible forms (labels and keyboard navigation), and show errors next to each field.

8. Suggested roadmap

Each phase ends with a system you can demo. The most important foundations come first.

Phase	Goal	Tasks	Result to demo
1. Clean up (2-3 days)	Fix what is broken	Delete SocialLoginPage. Fix the security items S1-S7. Move secrets to .env. Remove dist from the repo. Generic auth messages. Styling decision.	A clean, honest codebase
2. Worldwide data model (4-5 days)	Airports, time zones, fares	Migrations from Section 5.1. Seed about 30 international airports, routes, 2-3 aircraft types with real seat layouts, and fares in several currencies.	Search DAC to LHR and see fares per cabin in local times
3. Search that feels like an airline (3-4 days)	Return trips and connections	Return search, sp_search_itineraries for 1 stop, filters and sorting, fare calendar, airport type-ahead.	Return trip with a connection via DXB
4. Seat map + hold (4-5 days)	The Railway-inspired core	sp_hold_seats, release job, seat map API and page, countdown timer, traveller details with the domestic (passport/NID) and international (passport) document rules .	Two browsers race for one seat, and only one gets it
5. Payment + tickets (4-5 days)	Complete a purchase	Stripe test mode or a mock gateway, idempotency, sp_confirm_booking, PNR and e-ticket PDF, confirmation email (Mailpit).	Search through to an emailed e-ticket
6. After booking (4-5 days)	Manage, cancel, check in	Manage Booking by PNR, cancel/change under the fare rules, online check-in + QR boarding pass, flight status page.	Check in and show the boarding pass on a phone
7. Admin, reports, polish (4-5 days)	Operations + quality	Admin CRUD, reports from stored procedures, triggers + audit log, loyalty points, languages/currencies, PHPUnit tests, updated README + ER diagram.	Presentation-ready

Phase	Goal	Tasks	Result to demo
8. Growth features (4-6 days, optional)	Ideas from Section 9	Price history + price graph + price alerts, price freeze, date grid, 'Explore / Everywhere' search, EMI and payment-method offers, CO2 estimate.	Set a price alert and receive the email when the fare drops

8.1 Tests worth writing

- **Concurrency:** two parallel holds on the same seat. Exactly one succeeds, and the other gets HTTP 409.
- **Hold expiry:** after the hold time passes, the seat is available again and the booking is 'expired'.
- **Booking rules:** 10 passengers is rejected. More infants than adults is rejected. A passport that expires before travel is rejected.
- **Domestic documents:** DAC to CGP with an NID succeeds. DAC to DXB with an NID is rejected. A return trip with one international leg requires a passport.
- **Time zones:** a DAC to LHR flight shows the correct local departure and arrival, including a '+1 day' arrival.
- **Fare rules:** cancelling a Saver fare refunds only the taxes. A Flex fare refunds everything.
- **Auth:** a users-table account with role='admin' cannot open /admin routes.

8.2 Presentation tips

- State the idea clearly: *"We borrowed the Railway e-ticket platform's fairness features (seat map, timed hold, instant QR tickets) and built them into an international airline booking system."*
- Live demo of the seat race: open two browsers, both select the same seat, and only one gets it. Show the seat_holds rows in SSMS.
- Show the triggers working: set a flight to 'landed', then show the aircraft hours and the automatic maintenance log.
- Show the document rule: book Dhaka to Cox's Bazar with an NID (it works, and no visa is asked for), then try Dhaka to Dubai with the same NID (the database rejects it).
- Show an ER diagram of the final schema (SSMS Database Diagrams or dbdiagram.io).

9. More features: ideas from leading flight booking platforms

I looked at well-known flight search and booking platforms, both global and in Bangladesh, and noted the features travellers value most. The table lists each idea and **how it would fit into AeroBook's existing design**. Many of them reuse tables already planned in Section 5 (fares, seat_holds, routes, loyalty points).

Platform	Feature	How to add it to AeroBook	Effort
Google Flights	Price graph & date grid: a bar chart of prices over the coming weeks, and a colour-coded grid of departure x return dates	New fare_history table filled by trg_fare_history. A PIVOT query builds the grid. Chart with Recharts/Chart.js.	Medium
Google Flights	Price tracking alerts by email when a route's fare changes	price_alerts(user, route, date, target_price). A scheduled job compares the lowest current fare and sends an email.	Medium
Google Flights	'Prices are low / typical / high' insight	Compare today's fare with the route's history in fare_history (e.g. bottom 25% = low).	Low

Platform	Feature	How to add it to AeroBook	Effort
Google Flights	Emissions estimate and a lower-emissions filter	Estimated CO2 = routes.distance_km x a factor per cabin. Show it on each result, with a sort option.	Low
Google Flights / Skyscanner	Explore map / 'Everywhere' search: cheapest destinations from your home airport	One GROUP BY query (below), plus a Leaflet map using airports.lat/lon.	Low-Medium
Skyscanner	Whole-month view to find the cheapest day	Reuse the fare calendar endpoint with a month range.	Low
Kayak	Price forecast (buy now or wait)	A simple rule based on the recent trend in fare_history and the days left before departure. Label it 'likely to rise' or 'stable'.	Medium
Kayak	Mix-and-match one-ways ('Hacker Fares')	In a return search, also offer the cheapest outbound + cheapest return fares if together they cost less.	Low
Hopper	Price Freeze: pay a small fee to lock a fare for several days	A natural extension of the seat hold: fare_freezes(booking, locked_price, fee, expires_at). The fee counts towards the ticket.	Medium
Kiwi.com	Automatic check-in and real-time flight information	When check-in opens, a scheduled job checks travellers in and emails the boarding pass.	Low
Trip.com	Loyalty currency ('Trip Coins') redeemable on future bookings	Use the existing frequent_flyer_points: earn on sp_confirm_booking, and spend at checkout as a discount.	Low
Trip.com / Expedia	Flight + hotel bundles , airport transfers	Out of scope for now. Could be added later as partner links.	High
Expedia	Flexible tickets (free change within a date range)	Already covered by the Flex fare family. Add an optional 'cancel for any reason' extra.	Low
GoZayaan / ShareTrip	EMI (instalments) on bank credit cards (e.g. up to 12 months, minimum BDT 5,000)	AeroBook already has an 'EMI Policy' page with nothing behind it. Add emi_plans(bank, months, min_amount) and an EMI option at payment.	Low-Medium
GoZayaan / ShareTrip	Bank and mobile-banking offers (e.g. a discount on domestic flights with selected cards or wallets)	Extend promo_codes with payment_method, is_domestic_only and max_discount.	Low
Most platforms	Help centre + support chat , special assistance (wheelchair, special meal), group requests	support_tickets and special_requests tables, an FAQ page, and a group-request form for more than 9 travellers.	Low

Example: 'Everywhere' search (Skyscanner-style) in a single query

```
-- cheapest fare to every destination from the traveller's home airport in a given month
SELECT d.iata, d.city, d.country_code, MIN(fa.base_fare) AS from_price, r.is_domestic
FROM dbo.flights f
JOIN dbo.routes r ON r.id = f.route_id
JOIN dbo.airports d ON d.id = r.destination_airport_id
JOIN dbo.fares fa ON fa.flight_id = f.id
WHERE r.origin_airport_id = @home_airport_id
AND f.departure_utc >= @month_start AND f.departure_utc < DATEADD(MONTH, 1, @month_start)
AND f.status = 'scheduled'
GROUP BY d.iata, d.city, d.country_code, r.is_domestic
ORDER BY from_price; -- convert with exchange_rates before comparing mixed currencies
```

9.1 Recommended picks for this project

Not every idea is worth the time. These give the most value, and most of them also showcase database skills:

- **1. Price history, price graph and price alerts** (Google Flights). A trigger records every fare change, and a scheduled job sends the alerts. It shows triggers, scheduled jobs and email all in one feature.
- **2. Price Freeze** (Hopper). It reuses the seat-hold design, so it is a small extra step with a big 'wow' factor.
- **3. Date grid and fare calendar** (Google Flights / Skyscanner). A good use of PIVOT and GROUP BY queries.
- **4. EMI and bank/wallet offers** (GoZayaan / ShareTrip). They fit the Bangladeshi market, and the EMI Policy page already exists.
- **5. Explore / Everywhere search** with a map. A single query and a very visual demo.
- **6. CO2 estimate and loyalty point redemption.** Both are quick to add, because the data (distance, points) is already planned.

Better to skip for a student project: self-transfer itineraries across different airlines (Kiwi.com), hotel and car bundles (Trip.com / Expedia), and machine-learning price prediction. Each needs outside data sources or partner agreements, and together they would double the scope.

Bottom line: AeroBook has a solid foundation (a safe seat lock, clean SQL, working authentication). Today it is a one-way domestic search with a one-click reserve button. To become a worldwide flight platform it needs **international airports and time zones, fares with currencies, return and connecting itineraries, and per-booking traveller documents.** That includes the **domestic rule:** no visa, and a passport or NID. The Railway site contributes the **seat map, timed seat hold, instant QR ticket, booking lookup and anti-bot protection.** The leading booking platforms add **price tracking, price freeze, flexible-date search and local payment options.** Phases 2 to 5 are the essential core, and Phase 8 is where the extra polish comes from.

References

Bangladesh Railway e-ticketing: <https://eticket.railway.gov.bd/> (used only as a source of feature ideas)

Google Flights features (price graph, date grid, Explore, alerts, emissions): <https://www.going.com/guides/how-to-use-google-flights/> ; <https://upgradedpoints.com/travel/how-to-use-google-flights/>

Skyscanner, Kayak and Hopper features (Everywhere, Price Forecast, Hacker Fares, Price Freeze): <https://everything-pr.com/best-flight-comparison-sites/> ; <https://www.going.com/guides/best-apps-for-booking-flights>

Kiwi.com, Trip.com and Expedia: <https://thepointsguy.com/airline/use-kiwi-book-multi-ticket-travel/> ; <https://travelertribes.com/kiwi-com-vs-trip-com-which-is-better-for-flight/> ; <https://www.expedia.com/lp/b/flexible-flight-deals>

GoZayaan / ShareTrip (EMI, domestic flight offers): <https://gozayaan.com/emi-policy/> ; <https://gozayaan.com/campaign/id/560/> ; <https://sharetrip.net/>